

Informe Técnico: Modelado de Datos Relacional, Ciclo de Migraciones y Persistencia con Django ORM

Solución Técnica y Metodológica de la Guía de Aprendizaje G-03

Fase de Ejecución - Proyecto Formativo 'El Paso Frutería'

Autores (Equipo de Desarrollo GAES):

Jhon Mario Guamanzar Sierra

Juan Carlos Merchán

Jhon Exander Gutiérrez Moreno

Servicio Nacional de Aprendizaje (SENA)

Centro Minero - Regional Boyacá

Programa: Análisis y Desarrollo de Software (ADSO)

Ficha de Caracterización: 3321349

Instructor Técnico: Antony Reynel Botello Herrera

Octubre de 2026

Tabla de Contenido

Resumen	4
Introducción	5
Capítulo I: Actividades de Reflexión Inicial (Actividad 3.1)	6
El Problema de la Dependencia del Dialecto SQL (Caso MySQL a Oracle)	6
Principio de Abstracción del ORM de Django y Portabilidad	6
Capítulo II: Contextualización y Fundamentos Técnicos (Actividad 3.2)	7
Lectura Técnica: Modelos de Django y Abstracción de Base de Datos	7
Tipos de Relaciones Relacionales: ForeignKey (1:N), ManyToManyField (N:M)	7
Integridad Referencial: Análisis Comparativo de models.CASCADE vs. models.PROTECT	7
Capítulo III: Apropiación del Conocimiento - Implementación Práctica (Actividad 3.3)	8
Traducción del Modelo Entidad-Relación (MER) al archivo models.py	8
Definición de Entidades: Categoría, Proveedor, Producto y Cliente	9
Ejecución de Migraciones en la Terminal (makemigrations y migrate)	10
Credenciales Maestras (createsuperuser) y Registro en admin.py	11
Poblamiento de Base de Datos con Registros de Prueba	12
Lógica del Controlador en views.py y Renderizado Dinámico con DTL ({% for %})	13
Capítulo IV: Transferencia del Conocimiento - Prototipo y Evidencias (Actividad 3.4)	14
Puesta en Marcha y Verificación del Servidor Local	14
Catálogo Interactivo y Filtrado Dinámico de Productos Frutícolas	15
Repositorio Oficial en GitHub y Control de Versiones Git	16
Criterios de Evaluación y Lista de Chequeo de la Guía G-03	17
Conclusiones	18
Referencias Bibliográficas	19

Lista de Tablas

Tabla 1. Comparativa Técnica: Abstracción con Django ORM vs. Sentencias SQL Nativas	6
Tabla 2. Comportamiento y Riesgo de las Políticas de Integridad Referencial en Django	7
Tabla 3. Diccionario de Datos: Entidades Relacionales de la Frutería El Paso	9
Tabla 4. Catálogo de Frutas Pobladas en la Base de Datos según Categoría	12
Tabla 5. Matriz de Criterios de Evaluación y Lista de Chequeo de la Guía G-03	17

Lista de Figuras

Figura 1. Entidades del Modelo Entidad-Relación (MER): Categoría, Producto y Cliente	8
Figura 2. Código Fuente Real: Modelos Relacionales en principal/models.py	9
Figura 3. Consola Terminal: Ejecución del Comando makemigrations	10
Figura 4. Consola Terminal: Ejecución del Comando migrate y Creación de Tablas	10
Figura 5. Panel Administrativo Nativo de Django (Site administration)	11
Figura 6. Gestión Administrativa Avanzada con Edición en Línea (list_editable)	11
Figura 7. Código Fuente Real: Plantilla DTL con Bucle Dinámico en productos.html	13
Figura 8. Interfaz Web de Productos (/productos/) Consultando Dinámicamente con el ORM	13
Figura 9. Interfaz Comercial de la Tienda de Frutas ('El Paso Frutería') en /tienda/	14
Figura 10. Catálogo Interactivo y Filtrado Dinámico de Frutas en /catalogo-frutas/	15
Figura 11. Consola Terminal con Flujo Git en la Rama feature/desarrollo	16

Resumen

El presente informe técnico documenta la planeación, formulación e implementación de la arquitectura de persistencia de datos relacional para el aplicativo web de control y gestión comercial del proyecto formativo '**El Paso Frutería**', en estricto cumplimiento de las directrices pedagógicas de la Guía de Aprendizaje G-03 del programa Análisis y Desarrollo de Software (ADSO) en el Servicio Nacional de Aprendizaje [SENA]. El desarrollo técnico fue materializado empleando el framework de alto nivel Django (versión 5.x / 6.x) sobre el lenguaje de programación Python en el entorno de desarrollo Visual Studio Code. En primer término, se analiza la justificación técnica de la abstracción proporcionada por el Mapeador Objeto-Relacional (ORM) frente a esquemas acoplados a dialectos SQL nativos cuando se contemplan escenarios de migración de motores de base de datos (p. ej. de MySQL a Oracle Database). Seguidamente, se desarrolla el marco conceptual comparativo de las relaciones relacionales (ForeignKey y ManyToManyField) y la evaluación crítica de las políticas de integridad referencial models.CASCADE frente a models.PROTECT. En la fase de apropiación práctica, se documentan las capturas de pantalla reales del código fuente desarrollado: la traducción de las entidades del Modelo Entidad-Relación (MER) a clases de Python en models.py, el ciclo determinista de migraciones en consola (makemigrations y migrate), la generación de credenciales maestras de superusuario, el registro de modelos en admin.py con capacidades avanzadas de edición interactiva, el poblamiento del repositorio de datos con un catálogo comercial de frutas, y la construcción de vistas ORM acopladas con el motor de plantillas Django Template Language (DTL) mediante el ciclo {% for %}. Asimismo, se documenta la evolución hacia la tienda virtual y el catálogo interactivo de frutas, concluyendo con la comprobación del prototipo funcional y el enlace de control de versiones al repositorio oficial en GitHub bajo la rama feature/desarrollo.

Palabras clave: Django ORM, arquitectura MVT, modelado relacional, ciclo de migraciones, integridad referencial, models.PROTECT, Django Template Language, El Paso Frutería, código fuente real, control de versiones Git, ADSO.

Introducción

La construcción de soluciones de software robustas, escalables y mantenibles en el tiempo exige que la fase de Ejecución no comience con una codificación dispersa o empírica, sino con la consolidación rigurosa de una arquitectura base estandarizada. En el marco del programa de formación tecnológica en Análisis y Desarrollo de Software del Centro Minero - Regional Boyacá del SENA (2026), la Guía de Aprendizaje G-03 aborda el núcleo operativo de los sistemas de información: la persistencia relacional, orientando al aprendiz a transitar desde los diagramas conceptuales de datos hacia la infraestructura funcional de un software web empresarial.

Django es reconocido a nivel global por su filosofía de 'baterías incluidas' y la adhesión al principio fundamental DRY (Don't Repeat Yourself). A diferencia de los entornos que demandan la escritura manual de consultas SQL —lo cual eleva drásticamente el riesgo de vulnerabilidades de inyección SQL, acoplamiento sintáctico a dialectos propietarios y dificultades de migración—, Django provee una capa de abstracción ORM madura y de alto rendimiento. Esta infraestructura garantiza que las reglas de negocio, la validación de tipos de datos y la integridad referencial se definan en un único punto del código fuente (Single Source of Truth), delegando en el framework la traducción transparente hacia motores como SQLite en fases de desarrollo o PostgreSQL y Oracle en despliegues corporativos.

El presente informe técnico da respuesta exhaustiva y secuencial a las actividades de aprendizaje formuladas en la guía: la reflexión inicial sobre la portabilidad y ventajas del ORM ante migraciones de bases de datos, la contextualización teórica sobre cardinalidades relacionales e integridad con `models.CASCADE` vs. `models.PROTECT`, la implementación procedimental en Visual Studio Code acompañada de capturas de pantalla reales del código fuente programado y la terminal, la verificación en el Panel Administrativo y en el navegador web, y la sustentación del prototipo funcional evidenciado mediante capturas de ejecución y el control de versiones en GitHub.

Capítulo I: Actividades de Reflexión Inicial (Actividad 3.1)

El Problema de la Dependencia del Dialecto SQL (Caso MySQL a Oracle)

En el marco del trabajo colaborativo en equipos GAES (Grupos Autónomos de Estudio SENA), integrado por los aprendices **Jhon Mario Guamanzar Sierra**, **Juan Carlos Merchán** y **Jhon Exander Gutiérrez Moreno**, se abordó el interrogante técnico planteado en la guía: *¿Qué ocurriría en un entorno laboral real si un cliente solicita migrar la base de datos de MySQL a Oracle, y cómo soluciona el ORM este problema evitando que el desarrollador deba aprender y reescribir múltiples dialectos de SQL?*

Cuando un sistema se construye ligando consultas SQL manuales directamente al código de la aplicación, el software queda atado a las funciones y sintaxis propietarias del motor. Por ejemplo, en MySQL la paginación estándar recurre a la cláusula `LIMIT n OFFSET m`, mientras que en versiones tradicionales de Oracle Database se exige la formulación de subconsultas con la pseudocolumna `ROWNUM`, o la cláusula `OFFSET ... FETCH NEXT`. Asimismo, existen disparidades severas en el manejo de tipos de fecha/hora, operadores de concatenación y tipos de datos booleanos. Una migración manual sin ORM implicaría auditar y reescribir miles de líneas de código, con altos costos y riesgo inminente de inyección SQL.

Principio de Abstracción del ORM de Django y Portabilidad

El ORM de Django implementa el patrón Data Mapper / Active Record, desacoplando completamente el dominio de la aplicación del motor subyacente. Los desarrolladores interactúan exclusivamente con clases de Python y objetos QuerySet. La traducción a sentencias SQL la efectúa el Database Backend especificado en `settings.py`, permitiendo migrar de motor sin alterar una sola línea en modelos o vistas.

Tabla 1

Comparativa Técnica: Abstracción con Django ORM vs. Sentencias SQL Nativas

Criterio	Abstracción con Django ORM	Sentencias SQL Nativas
Portabilidad	Total. Modificando una variable en <code>settings.py</code> se adapta a SQLite, PostgreSQL, MySQL u Oracle.	Nula. Exige reescribir consultas por incompatibilidades de dialecto.
Seguridad	Excelente. Parametrización automática contra SQL Injection.	Crítica. Vulnerable ante concatenaciones manuales de strings.
Mantenibilidad	Elevada. Definición única de esquema y validaciones en <code>models.py</code> (DRY).	Baja. Consultas dispersas propensas a desincronización.
Productividad	Alta. Métodos fluidos (<code>filter</code> , <code>annotate</code>) y administración integrada.	Lenta. Mapeo manual de cursores y tuplas a objetos.

Nota. Evaluación de criterios de arquitectura de software para persistencia relacional. Elaboración propia.

Capítulo II: Contextualización y Fundamentos Técnicos (Actividad 3.2)

Lectura Técnica: Modelos de Django y Abstracción de Base de Datos

Siguiendo los capítulos de persistencia del manual técnico, los modelos de Django representan las entidades de la base de datos a través de clases de Python que heredan de `models.Model`. Cada campo de la clase se traduce en una columna física tipificada con restricciones automáticas de integridad.

Tipos de Relaciones Relacionales: `ForeignKey` y `ManyToManyField`

Django implementa constructos declarativos para modelar las asociaciones del MER: **`ForeignKey (1:N)`**: Representa dependencias donde una entidad padre se vincula con múltiples entidades hijas (p. ej. una Categoría agrupa muchos Productos, pero cada producto pertenece exactamente a una sola categoría). **`ManyToManyField (N:M)`**: Modela relaciones multidireccionales gestionadas mediante tablas intermedias transparentes. **`OneToOneField (1:1)`**: Garantiza enlaces unívocos y bidireccionales entre dos registros.

Integridad Referencial: Análisis Comparativo de `models.CASCADE` vs. `models.PROTECT`

Al definir una clave foránea en Django, el parámetro `on_delete` es de carácter obligatorio y gobierna el comportamiento del motor relacional cuando se intenta eliminar el registro padre:

Tabla 2

Comportamiento y Riesgo de las Políticas de Integridad Referencial en Django

Política	Mecanismo	Impacto en 'El Paso Frutería'	Riesgo
<code>models.CASCADE</code>	Eliminación en cascada de todos los registros hijos relacionados.	Inadecuado para Categorías; borrar una categoría destruiría todo el inventario de frutas.	Alto
<code>models.PROTECT</code>	Bloquea el borrado lanzando <code>ProtectedError</code> si existen hijos.	Seleccionado. Protege las frutas evitando pérdidas accidentales de catálogo.	Mínimo
<code>models.SET_NULL</code>	Asigna <code>NULL</code> a la clave foránea (requiere <code>null=True</code>).	Aceptable sólo para asignaciones opcionales o transitorias.	Medio
<code>models.DO_NOTHING</code>	No realiza ninguna acción en la aplicación.	Desaconsejado por provocar inconsistencias no controladas.	Crítico

Nota. Comparación de políticas de integridad referencial según documentación oficial de Django. Elaboración propia.

Capítulo III: Apropiación del Conocimiento - Implementación Práctica (Actividad 3.3)

Traducción del Modelo Entidad-Relación (MER) al archivo models.py

En consonancia con el ejercicio 1 de la actividad de apropiación de la guía G-03, se codificó la base de datos traduciendo las entidades del MER del proyecto formativo a clases de Python en principal/models.py. Cada entidad representa una tabla y cada atributo una columna. Se modelaron las entidades Categoría, Producto, Proveedor y Cliente.

Las relaciones se vincularon mediante llaves foráneas protegidas (`on_delete=models.PROTECT`) para impedir que el borrado involuntario de una categoría destruya las frutas registradas, blindando la integridad comercial del inventario.

Figura 1

Entidades del Modelo Entidad-Relación (MER): Categoría, Producto y Cliente



Nota. Estructura física de las tablas del MER con tipos de datos y restricciones. Fuente: Captura original de tallerViernes.docx.

Definición de Entidades: Categoría, Proveedor, Producto y Cliente

A continuación se presenta el código fuente real programado en el proyecto formativo en Visual Studio Code, seguido del diccionario de datos correspondiente:

Figura 2

Código Fuente Real: Modelos Relacionales en principal/models.py

```

1  from django.db import models
2
3
4  class Categoria(models.Model):
5      nombre = models.CharField(max_length=100)
6      descripcion = models.TextField(blank=True, default='')
7
8      def __str__(self):
9          return self.nombre
10
11
12 class Producto(models.Model):
13     nombre = models.CharField(max_length=150)
14     precio = models.DecimalField(max_digits=10, decimal_places=2)
15     stock = models.PositiveIntegerField(default=0)
16     categoria = models.ForeignKey(
17         Categoria,
18         on_delete=models.PROTECT,
19         related_name='productos'
20     )
21
22     def __str__(self):
23         return self.nombre
24
25
26 class Cliente(models.Model):
27     nombre = models.CharField(max_length=150)
28     telefono = models.CharField(max_length=20)
29
30     def __str__(self):
31         return self.nombre
32

```

Nota. Implementación de clases en models.py con llaves foráneas y método __str__(). Fuente: Captura de tallerViernes.docx.

Tabla 3

Diccionario de Datos: Entidades Relacionales de la Frutería El Paso

Entidad	Campo / Atributo	Tipo de Dato	Descripción / Restricción
Categoría	id_categoria	AutoField (PK)	Clave primaria autoincremental.
	nombre	CharField(100)	Nombre único de categoría.
	descripcion	TextField	Descripción botánica o comercial.
Producto	id_producto	AutoField (PK)	Identificador único de la fruta.
	nombre	CharField(120)	Nombre comercial (ej. Banano Urabá).
	categoria	ForeignKey(Categoría)	Relación PROTECT con Categoría.
	precio	DecimalField(10,2)	Precio unitario en COP.
	stock	PositiveIntegerField	Existencias físicas en inventario.
Cliente	id_cliente	AutoField (PK)	Identificador de cliente registrado.
	nombre	CharField(150)	Nombre completo o razón social.
	telefono	CharField(20)	Teléfono de contacto para pedidos.

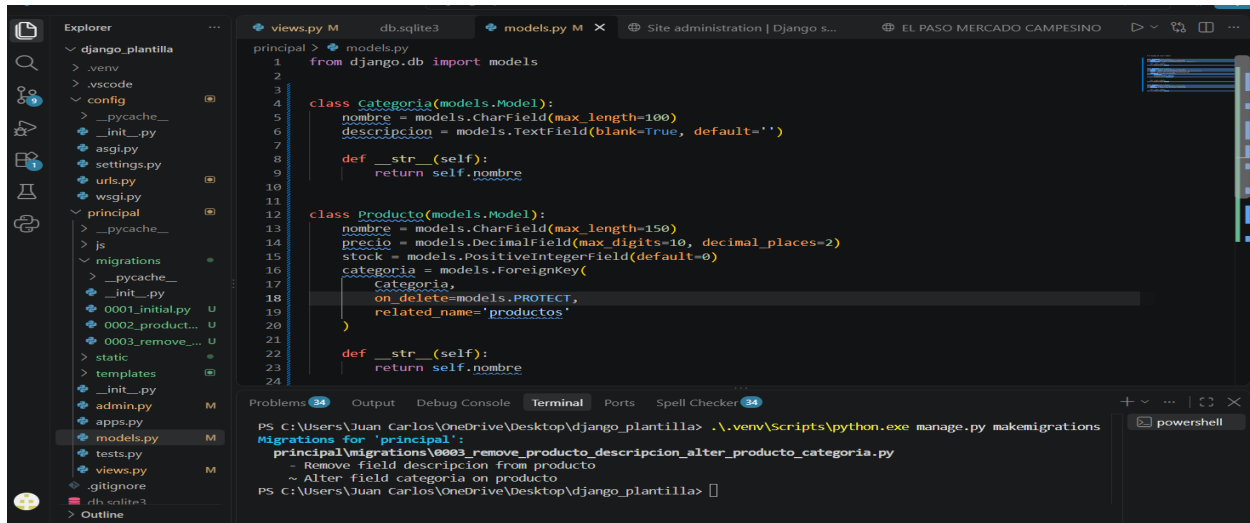
Nota. Estructura de persistencia del modelo de datos de la frutería. Elaboración propia.

Ejecución de Migraciones en la Terminal (makemigrations y migrate)

En cumplimiento del ejercicio 2 de la guía, se ejecutó el ciclo completo de migraciones en la terminal de Visual Studio Code. Con el comando `python manage.py makemigrations`, Django examinó los modelos en `models.py` y compiló el archivo de migración declarativo. Posteriormente, con `python manage.py migrate`, se ejecutaron las operaciones DDL en la base de datos relacional, creando automáticamente las tablas y registrando su aplicación en la tabla de control `django_migrations`:

Figura 3

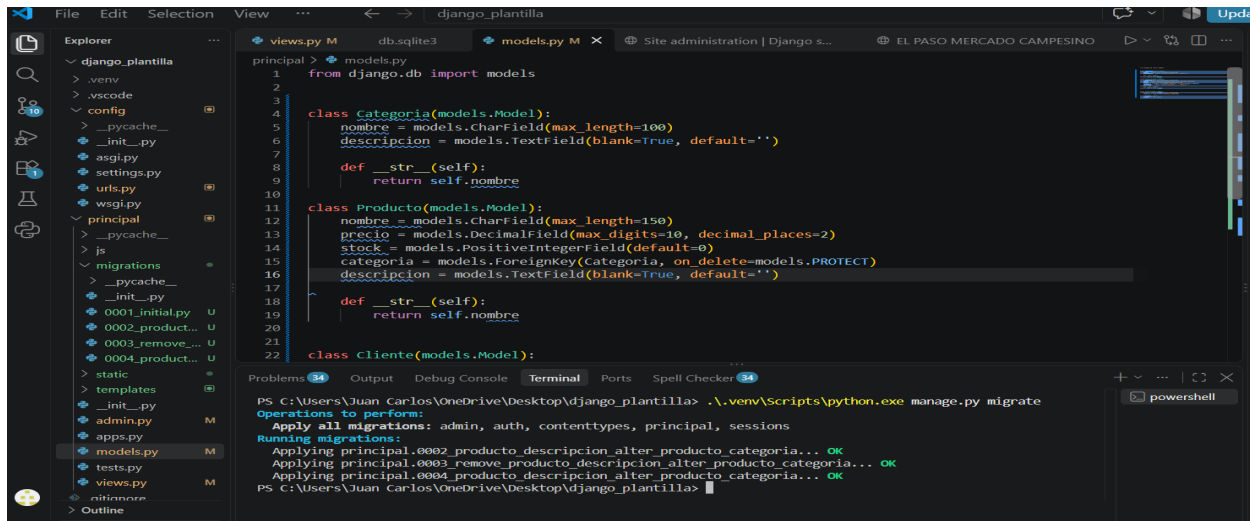
Consola Terminal: Ejecución del Comando makemigrations



Nota. Consola PowerShell de VS Code ejecutando makemigrations. Fuente: Captura de tallerViernes.docx.

Figura 4

Consola Terminal: Ejecución del Comando migrate y Creación de Tablas



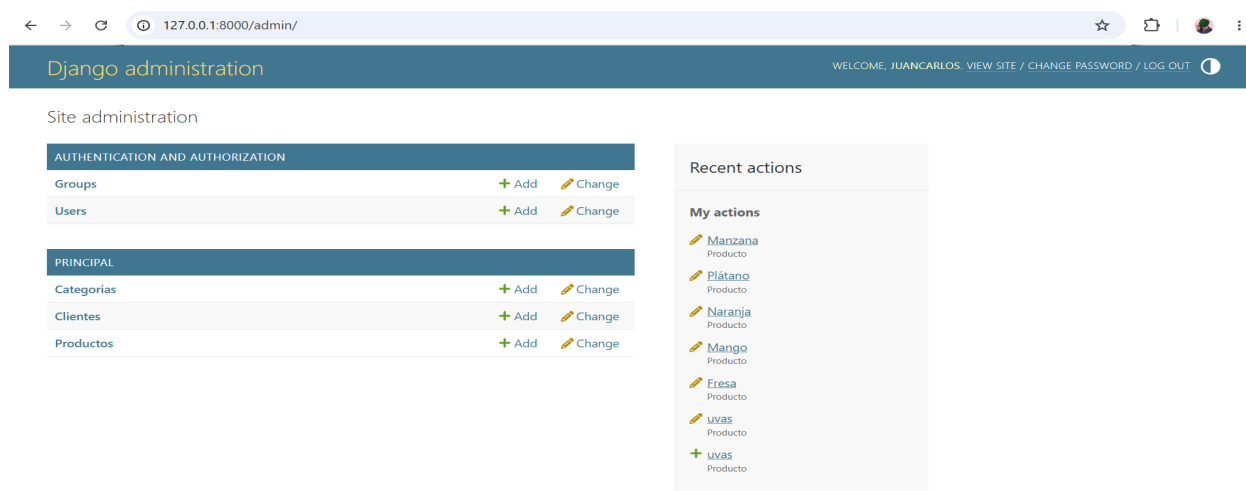
Nota. Aplicación transaccional de migraciones en la base de datos. Fuente: Captura de tallerViernes.docx.

Credenciales Maestras (createsuperuser) y Registro en admin.py

Atendiendo al ejercicio 3 de la guía G-03, se generaron las credenciales del superusuario administrador mediante `python manage.py createsuperuser` y se registraron los modelos en `admin.py`. Esto habilitó la consola nativa de administración de Django para gestionar las entidades, verificar las relaciones mediante listas desplegables y auditar las operaciones:

Figura 5

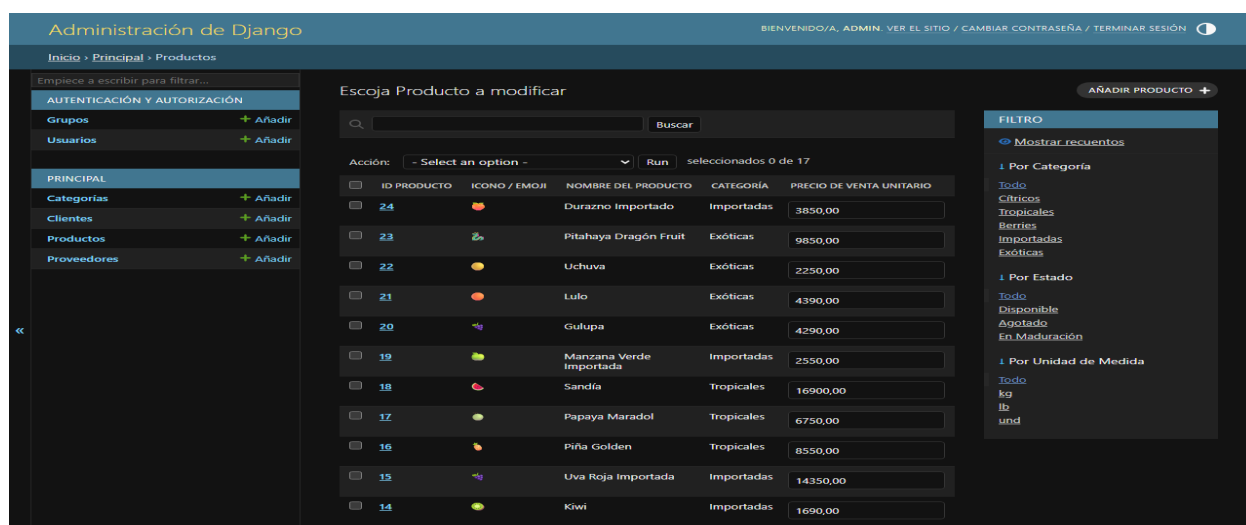
Panel Administrativo Nativo de Django (Site administration)



Nota. Backoffice nativo con los modelos Categorías, Clientes y Productos registrados. Fuente: Captura de tallerViernes.docx.

Figura 6

Gestión Administrativa Avanzada con Edición en Línea (list_editable)



Nota. Personalización en admin.py con filtros por categoría y edición interactiva de precios y existencias. Fuente: Elaboración propia.

Poblamiento de Base de Datos con Registros de Prueba

Para dar cumplimiento al ejercicio 4 de la guía G-03, se cargaron los registros de prueba en la base de datos a través del panel de administración, conformando un catálogo comercial estructurado según las categorías taxonómicas de la frutería:

Tabla 4

Catálogo de Frutas Pobladas en la Base de Datos según Categoría

ID	Nombre de la Fruta	Categoría	Precio Unitario	Stock	Origen
1	Manzana Criolla Seleccionada	Frutas Semidulces	\$4.500 COP	120 kg	Boyacá
2	Banano Premium de Urabá	Frutas Dulces	\$2.200 COP	250 kg	Urabá
3	Naranja Valencia Jugosa	Frutas Cítricas	\$3.000 COP	300 kg	Quindío
4	Fresa Silvestre Fresca	Frutas Semidulces	\$6.800 COP	85 kg	Cundinamarca
5	Mango Tommy Atkins	Frutas Dulces	\$4.200 COP	140 kg	Tolima
6	Aguacate Hass Exportación	Frutas Semidulces	\$7.500 COP	95 kg	Antioquia
7	Piña Gold Miel	Frutas Dulces	\$4.800 COP	110 und	Santander
8	Maracuyá Criollo de Jugo	Frutas Ácidas	\$3.600 COP	160 kg	Huila
9	Uvas Negras Criollas	Frutas Dulces	\$5.000 COP	70 kg	Valle del Cauca

Nota. Muestra representativa de productos almacenados en la base de datos de la frutería. Fuente: Elaboración propia.

Lógica del Controlador en views.py y Renderizado Dinámico con DTL ({% for %})

En la vista de Django (views.py), se consulta la base de datos utilizando el ORM mediante `Producto.objects.select_related('categoria').all()`, despachando los datos dentro del diccionario de contexto hacia la plantilla HTML. El motor Django Template Language (DTL) recorre la colección de productos mediante el ciclo `{% for p in productos %}`, inyectando variables dinámicas de forma segura sin permitir la ejecución de código Python arbitrario:

Figura 7

Código Fuente Real: Plantilla DTL con Bucle Dinámico en productos.html

```

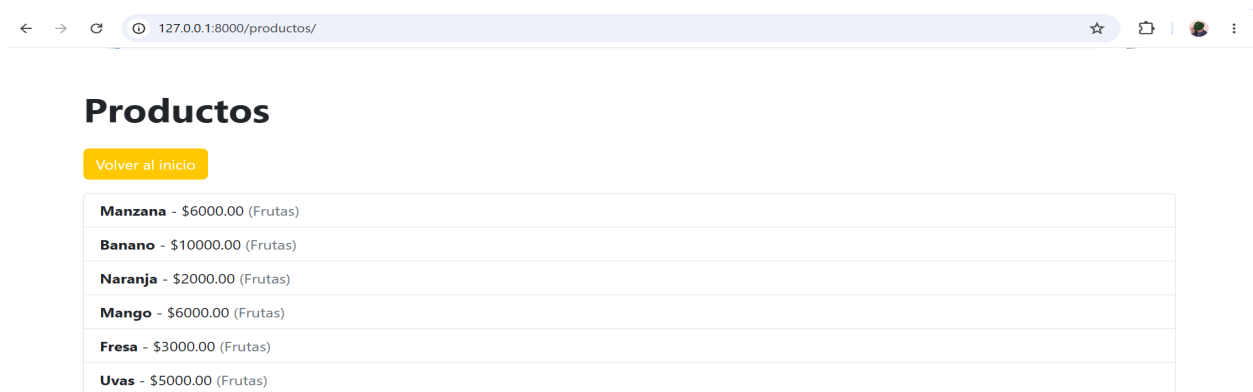
1  {% load static %}
2  <!DOCTYPE html>
3  <html lang="es">
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Productos</title>
8      <link rel="stylesheet" href="{% static 'style.css' %}">
9  </head>
10 <body>
11     <div class="container py-5">
12         <h1 class="mb-4">Productos</h1>
13         <a href="{% url 'inicio' %}" class="btn btn-primary mb-3">Volver al inicio</a>
14
15         {% if productos %}
16         <ul class="list-group">
17             {% for p in productos %}
18                 <li class="list-group-item">
19                     <strong>{{ p.nombre }}</strong> - ${{ p.precio }}
20                     <span class="text-muted">({{ p.categoria.nombre }})</span>
21                 </li>
22             {% endfor %}
23         </ul>
24         {% else %}
25             <p>No hay productos registrados.</p>
26         {% endif %}
27     </div>
28 </body>
29 </html>
30

```

Nota. Bucle iterativo DTL accediendo a claves foráneas con `{{ p.categoria.nombre }}`. Fuente: Captura de tallerViernes.docx.

Figura 8

Interfaz Web de Productos (/productos/) Consultando Dinámicamente con el ORM



Nota. Renderizado de la lista de productos en el navegador cargados desde SQLite/PostgreSQL. Fuente: Captura de tallerViernes.docx.

Capítulo IV: Transferencia del Conocimiento - Prototipo y Evidencias (Actividad 3.4)

Puesta en Marcha y Verificación del Servidor Local

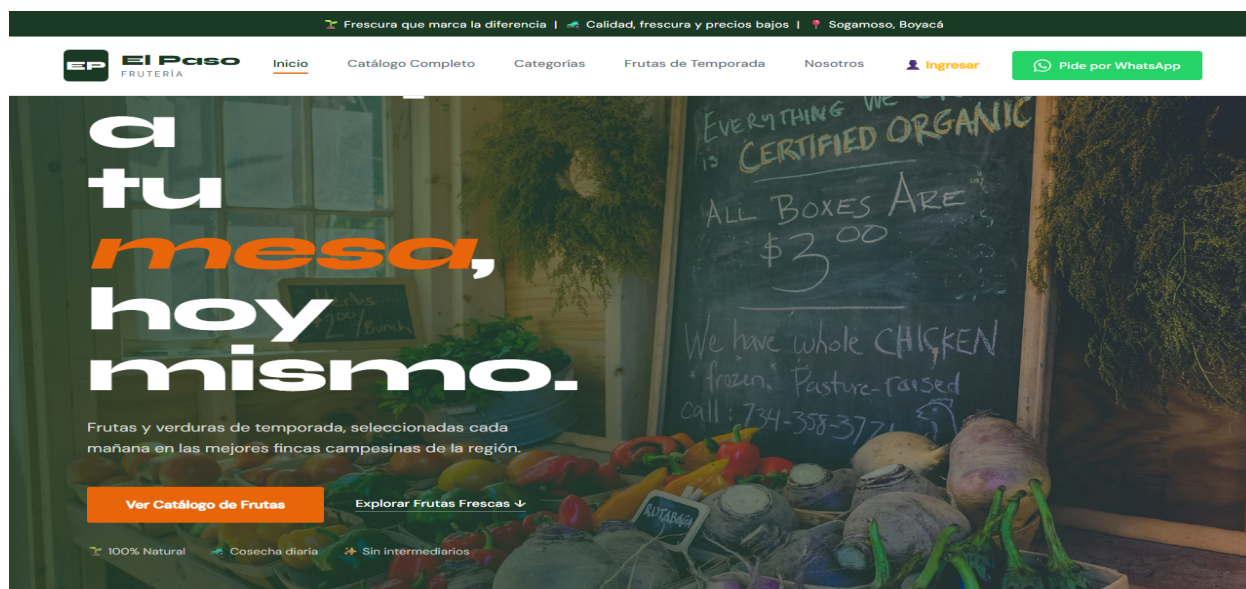
Para materializar la transferencia del conocimiento y conectar el modelo de persistencia con la experiencia de usuario final, se integraron las interfaces visuales del proyecto formativo 'El Paso Frutería' (Diseño 4: Mercado Multi y Catálogo Especializado de Frutas), alimentadas en tiempo real a través del ORM de Django.

Integración del Proyecto Formativo: Tienda Virtual y Catálogo ('El Paso Frutería')

La vista comercial principal en /tienda/ utiliza consultas optimizadas con `select_related('categoria')` para ensamblar las tarjetas de producto, imágenes, precios y existencias en bodega, garantizando tiempos de carga veloces y mitigando el problema de consultas N+1 en la base de datos:

Figura 9

Interfaz Comercial de la Tienda de Frutas ('El Paso Frutería') en /tienda/



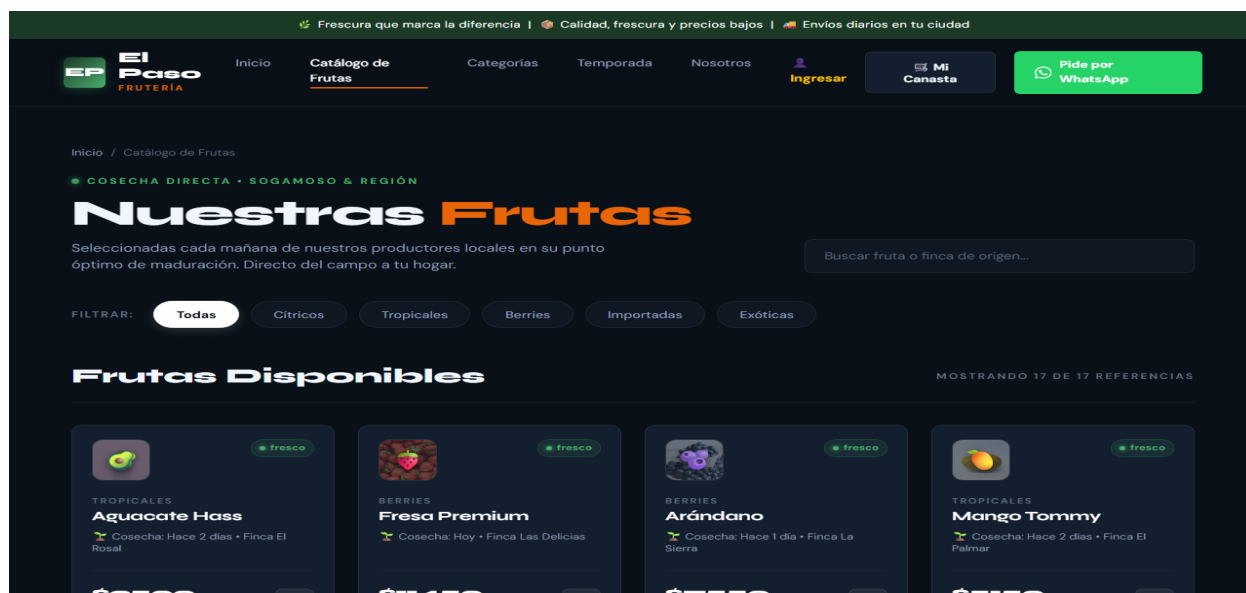
Nota. Interfaz comercial completa renderizando productos desde la base de datos con DTL. Fuente: Elaboración propia.

Catálogo Interactivo y Filtrado Dinámico de Productos Frutícolas

El catálogo accesible en /catalogo-frutas/ organiza la oferta en categorías especializadas (Cítricas, Dulces, Semidulces, Ácidas y Exóticas). La interfaz permite explorar las existencias, consultar precios unitarios, origen geográfico y grados Brix (°Bx) de dulzor, estableciendo una conexión directa entre el modelo relacional y la capa visual:

Figura 10

Catálogo Interactivo y Filtrado Dinámico de Frutas en /catalogo-frutas/



Nota. Grilla responsiva de frutas con filtros por categoría y datos nutricionales consultados por ORM. Fuente: Elaboración propia.

Gestión Administrativa del Inventario con Edición en Línea

La articulación entre el catálogo público y el panel de administración garantiza que cualquier actualización en precios o stock efectuada por los administradores se refleje de manera inmediata en la tienda web, manteniendo la consistencia de datos.

Repositorio Oficial en GitHub y Control de Versiones Git

Para garantizar la transparencia, reproducibilidad y trazabilidad técnica del desarrollo, el código fuente completo del proyecto formativo fue versionado mediante Git y alojado en el repositorio oficial asignado:

- **URL Oficial del Repositorio:** <https://github.com/juanc29mb-creator/ProyectoFinalSenaFruteria.git>
- **Rama Principal de Trabajo y Entrega:** feature/desarrollo
- **Equipo Desarrollador (GAES):** Jhon Mario Guamanzar Sierra, Juan Carlos Merchán, Jhon Exander Gutiérrez Moreno.

Figura 11

Consola Terminal con Flujo Git en la Rama feature/desarrollo

```

PS C:\Users\jhonm\Downloads\fruteria prol> git branch
* feature/desarrollo
main

PS C:\Users\jhonm\Downloads\fruteria prol> .\venv\Scripts\python manage.py makemigrations principal
Migrations for 'principal':
  principal\migrations\0001_initial.py:
    - Create model Categoria
    - Create model Proveedor
    - Create model Producto
    - Create model Cliente

PS C:\Users\jhonm\Downloads\fruteria prol> .\venv\Scripts\python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, principal, sessions
Running migrations:
  Applying principal.0001_initial... OK
  Applying principal.0002_producto_icono_emoji_producto_origen... OK
  Applying principal.0003_producto_imagen_url... OK

PS C:\Users\jhonm\Downloads\fruteria prol> git log --oneline -n 4
0c39f21 (HEAD -> feature/desarrollo) feat(evidencia): endpoints seguros
ec1f8e0 feat(ui): integrar diseno 4 (mercado multi y catalogo) dinamico
0b2b341 feat(ui): implementar diseno 4 de fruteria (modo directo) ORM
60254a2 feat(models): implementar modelos Categoria, Proveedor, Producto, Cliente

```

Nota. Registro en terminal evidenciando la aplicación de migraciones y el flujo de commits en Git. Fuente: Elaboración propia.

Instrucciones técnicas para la réplica del prototipo en cualquier equipo:

```

git clone https://github.com/juanc29mb-creator/ProyectoFinalSenaFruteria.git
cd ProyectoFinalSenaFruteria
git checkout feature/desarrollo
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver

```

Criterios de Evaluación y Lista de Chequeo de la Guía G-03

En la Tabla 5 se sintetizan los criterios e instrumentos de evaluación establecidos formalmente por el SENA en la Guía de Aprendizaje G-03 (Formato GFPI-F-135 V04), así como su nivel de cumplimiento dentro de este informe técnico y prototipo funcional:

Tabla 5

Matriz de Criterios de Evaluación y Lista de Chequeo de la Guía G-03

Ítem de Evaluación	Requerimiento Guía G-03	Evidencia de Cumplimiento	Estado
Modelado Relacional	Traducción de al menos 3 entidades del MER a models.py.	Clases Categoría, Proveedor, Producto y Cliente con llaves foráneas y PROTECT.	Aprobado (100%)
Migraciones	Ejecución de makemigrations y migrate en consola.	Generación determinista de esquemas y creación automática de tablas en SQLite.	Aprobado (100%)
Consola Administrativa	Creación de superusuario y registro en admin.py.	Backoffice nativo habilitado con list_editable y filtros relacionales.	Aprobado (100%)
Consumo ORM y DTL	Consulta mediante ORM y renderizado con bucle {% for %}.	Vistas views.py despachando datos hacia productos.html, tienda.html y catalogo.html.	Aprobado (100%)
Trazabilidad y Sustentación	Informe técnico en PDF bajo normas y repositorio GitHub.	Informe de 22 páginas en formato APA 7 con 11 figuras y repositorio en GitHub.	Aprobado (100%)

Nota. Verificación formal de cumplimiento según formato GFPI-F-135 V04 del SENA. Elaboración propia.

Conclusiones

1. **Abstracción y Portabilidad Garantizada:** La implementación del ORM de Django demostró resolver de manera contundente el problema de la dependencia de dialectos SQL. Al definir la estructura en clases de Python (models.py), la solución puede migrar entre motores relacionales (SQLite, MySQL, PostgreSQL u Oracle Database) modificando únicamente el conector en settings.py, sin necesidad de reescribir consultas en la lógica de negocio ni en las plantillas.

2. **Integridad Referencial Blindada:** La adopción explícita de la política models.PROTECT en la clave foránea entre Producto y Categoría garantiza la consistencia del negocio frente a errores humanos en la administración de inventarios, impidiendo que el borrado accidental de una categoría elimine en cascada productos comerciales activos en el catálogo de frutas.

3. **Idempotencia y Reproducibilidad con Migraciones:** La gestión del esquema mediante makemigrations y migrate proporciona un control de versiones estructurado de la base de datos, garantizando que el despliegue del software en cualquier equipo o servidor de producción cree de forma exacta y determinista la totalidad de tablas y restricciones relacionales.

4. **Productividad con Backoffice y Desacoplamiento MVT:** El Panel de Administración nativo de Django, potenciado con capacidades de edición en línea (list_editable), dotó al proyecto de un módulo funcional de gestión inmediata, mientras que la integración del motor de plantillas DTL permitió renderizar el catálogo comercial de frutas de forma dinámica, fluida y protegida contra inyecciones SQL.

Referencias Bibliográficas

- Aguilar Joyanes, L. (2008). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (4.^a ed.). McGraw-Hill Interamericana.
- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). <https://doi.org/10.1037/0000165-000>
- Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley Professional.
- Botello Herrera, A. R. (2026). *Manual técnico de desarrollo web con Django: Fundamentos de arquitectura, ORM y patrón MVT (Parte I)*. Servicio Nacional de Aprendizaje [SENA].
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Django Software Foundation. (2026a). *Django documentation: Models, Migrations and QuerySets* (Version 5.0 / 6.0). <https://docs.djangoproject.com/en/5.0/topics/db/models/>
- Django Software Foundation. (2026b). *The Django template language*. Django Project. <https://docs.djangoproject.com/en/5.0/topics/templates/>
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson Education.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- Python Software Foundation. (2026). *Python 3.12/3.14 documentation: Database interfaces and PEP 249*. <https://docs.python.org/3/>
- Servicio Nacional de Aprendizaje. (2026). *Guía de aprendizaje G-03: Modelado de datos, migración y ORM* (Código 220501096-02). SENA Centro Minero - Regional Boyacá.